Domain 1: Monitoring, Logging, Analysis, Remediation, and Performance Optimization

AWS Certified SysOps Administrator – Associate (SOA-C03)

> ***Exam Weight:*** *~20% of scored content*
> ***Source:*** *AWS Certified SysOps Administrator – Associate Exam Guide*

Overview

Domain 1 is the largest domain in the CloudOps Associate exam. It covers the full observability lifecycle: collecting metrics and logs, setting up alarms and dashboards, routing and responding to events, automating remediation, and optimizing the performance of compute, storage, and database resources. A CloudOps engineer must be comfortable operating across CloudWatch, CloudTrail, EventBridge, Systems Manager, and the performance tooling for EC2, EBS, S3, EFS, FSx, and RDS.

Task 1.1: Implement Metrics, Alarms, and Filters Using AWS Monitoring and Logging Services

Skill 1.1.1 – Configure AWS Monitoring and Logging (CloudWatch, CloudTrail, Amazon Managed Service for Prometheus)

Amazon CloudWatch

Amazon CloudWatch is the primary observability service for AWS. It collects and tracks metrics, collects and monitors log files, sets alarms, and automatically reacts to changes in your AWS resources.

> *Source: What is Amazon CloudWatch? – Amazon CloudWatch*

Core CloudWatch Concepts

Concept	Description
Namespace	A container for CloudWatch metrics (e.g., `AWS/EC2`, `AWS/RDS`). Custom metrics use a custom namespace.
Metric	A time-ordered set of data points published to CloudWatch. Each metric has a name, namespace, and optional dimensions.
Dimension	A name/value pair that uniquely identifies a metric (e.g., `InstanceId=i-1234567890abcdef0`).
Resolution	Standard metrics have 1-minute granularity; high-resolution custom metrics can be published at 1-second intervals.
Statistics	Aggregations over a period: Average, Sum, Minimum, Maximum, SampleCount, and percentiles (p99, p99.9).
Period	The length of time (in seconds) over which a statistic is applied.
Alarm	Watches a single metric and performs actions when the metric crosses a threshold.
Dashboard	A customizable home page in the CloudWatch console for monitoring resources in a single view.

CloudWatch Logs

CloudWatch Logs lets you centralize logs from EC2 instances, Lambda functions, CloudTrail, Route 53, and other sources. Key concepts:

- **Log group** – A collection of log streams that share the same retention, monitoring, and access control settings.
- **Log stream** – A sequence of log events from a single source (e.g., one EC2 instance).
- **Metric filter** – Extracts metric observations from log events and transforms them into CloudWatch metrics.
- **Subscription filter** – Delivers real-time log data to Kinesis Data Streams, Kinesis Data Firehose, or Lambda.
- **Log Insights** – An interactive query service for analyzing log data using a purpose-built query language.

CloudWatch Logs Insights Example Query

```
fields @timestamp, @message
  | filter @message like /ERROR/
  | sort @timestamp desc
  | limit 20
```

Retention Policies

By default, log groups never expire. Set a retention policy (1 day to 10 years) to control storage costs. Logs that exceed the retention period are automatically deleted.

AWS CloudTrail

AWS CloudTrail records API calls made in your AWS account, delivering log files to an S3 bucket. It provides governance, compliance, operational auditing, and risk auditing of your AWS account.

> *Source: What Is AWS CloudTrail? – AWS CloudTrail*

Event Types

Event Type	Description
Management events	Operations performed on AWS resources (e.g., `CreateBucket`, `RunInstances`, `AttachRolePolicy`). Enabled by default.
Data events	Resource-level operations (e.g., S3 `GetObject`, Lambda `Invoke`). High volume; disabled by default.
Insights events	Detect unusual API call rates or error rates. Requires explicit enablement.
Network activity events	Capture VPC endpoint network activity. Configured via advanced event selectors.

Trail vs. Event History

- **Event History** – Free, 90-day rolling view of management events in the console. Not configurable.
- **Trail** – Delivers events to S3 (and optionally CloudWatch Logs and SNS). Supports multi-Region and organization-wide coverage.

Creating a Multi-Region Trail (Best Practice)

```
aws cloudtrail create-trail \
    --name my-org-trail \
    --s3-bucket-name my-cloudtrail-bucket \
    --is-multi-region-trail \
    --include-global-service-events \
    --enable-log-file-validation
```

CloudTrail Lake

CloudTrail Lake is a managed data lake for CloudTrail events. It lets you run SQL-based queries directly on event data without exporting to S3 and querying with Athena. Event data stores can retain data for up to 7 years.

Security Best Practices

- Enable log file validation (`--enable-log-file-validation`) to detect tampering.
- Encrypt log files with SSE-KMS.
- Restrict S3 bucket access with a bucket policy; deny `s3:DeleteObject` for the trail bucket.
- Enable multi-Region trails to capture global service events (IAM, STS, CloudFront).

- Send CloudTrail logs to CloudWatch Logs for real-time alerting on sensitive API calls.

> *Source: Security best practices in AWS CloudTrail*

Amazon Managed Service for Prometheus (AMP)

Amazon Managed Service for Prometheus is a serverless, Prometheus-compatible monitoring service for container metrics. It is designed for Kubernetes workloads running on Amazon EKS, Amazon ECS, or self-managed Kubernetes clusters.

> *Source: Amazon Managed Service for Prometheus – User Guide*

Key Concepts

- **Workspace** – An isolated Prometheus environment. Each workspace has its own ingestion endpoint and query endpoint.
- **Scraper** – An agentless, AWS-managed component that automatically discovers and scrapes metrics from EKS clusters without deploying a Prometheus server.
- **PromQL** – The query language used to query metrics stored in AMP.
- **Alert Manager** – Handles deduplication, grouping, and routing of alerts generated by Prometheus rules.

Integration with Amazon Managed Grafana

AMP is commonly paired with **Amazon Managed Grafana** for visualization. Grafana connects to AMP as a data source using SigV4 authentication, enabling rich dashboards for Kubernetes workloads.

AMP vs. CloudWatch Container Insights

Aspect	Amazon Managed Service for Prometheus	CloudWatch Container Insights
Query language	PromQL	CloudWatch Metrics Insights / Log Insights
Primary use case	Kubernetes-native metrics	ECS, EKS, and EC2 container metrics
Visualization	Amazon Managed Grafana	CloudWatch dashboards
Alerting	Prometheus Alert Manager	CloudWatch Alarms

Skill 1.1.2 – Configure and Manage the CloudWatch Agent

The **CloudWatch agent** is a software package that runs on EC2 instances, on-premises servers, and container environments. It collects system-level metrics (CPU, memory, disk, network) and log files that are not available through the default EC2 hypervisor metrics.

> *Source: Collect metrics, logs, and traces using the CloudWatch agent – Amazon CloudWatch*

Why Use the CloudWatch Agent?

Default EC2 metrics (available without the agent) include CPU utilization, network I/O, and disk I/O at the hypervisor level. They do **not** include:

- Memory utilization
- Disk space utilization (used/free)
- Swap utilization
- Per-process metrics

The CloudWatch agent fills this gap by collecting OS-level metrics and forwarding them to CloudWatch under a custom namespace (default: `CWAgent`).

Installation Methods

Method	Description
Systems Manager Run Command	Use `AWS-ConfigureAWSPackage` to install the agent on managed instances at scale
Manual installation	Download and install the agent package directly on the instance
EC2 Image Builder	Bake the agent into a custom AMI using an Image Builder component
Container sidecar	Run the agent as a sidecar container in ECS or EKS

IAM Requirements

The EC2 instance profile (or task role for ECS/EKS) must include:

- `CloudWatchAgentServerPolicy` – Allows the agent to publish metrics and logs.
- `AmazonSSMManagedInstanceCore` – Required if using Systems Manager to distribute the configuration.

Agent Configuration File

The agent is configured via a JSON file (`/opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json` on Linux). The configuration wizard (`amazon-cloudwatch-agent-config-wizard`) can generate this file interactively.

```
{
    "metrics": {
      "namespace": "CWAgent",
      "metrics_collected": {
        "mem": {
          "measurement": ["mem_used_percent"]
        },
        "disk": {
          "measurement": ["disk_used_percent"],
          "resources": ["/", "/data"]
        }
      }
    },
    "logs": {
      "logs_collected": {
        "files": {
          "collect_list": [
            {
              "file_path": "/var/log/application.log",
              "log_group_name": "/myapp/application",
              "log_stream_name": "{instance_id}"
            }
          ]
        }
      }
    }
  }
```

Storing Configuration in Parameter Store

Store the agent configuration in **AWS Systems Manager Parameter Store** (e.g., under `AmazonCloudWatch-linux`) and distribute it to instances using the `AmazonCloudWatch-ManageAgent` SSM document. This enables centralized configuration management across a fleet.

CloudWatch Agent on ECS

For ECS clusters, deploy the CloudWatch agent as a **daemon service** (EC2 launch type) or as a **sidecar container** (Fargate). The agent collects container-level metrics and forwards them to CloudWatch Container Insights.

CloudWatch Agent on EKS

For EKS clusters, deploy the CloudWatch agent as a **DaemonSet** using the `amazon-cloudwatch-observability` add-on. For log collection, deploy **Fluent Bit** as a DaemonSet to forward container logs to CloudWatch Logs.

Troubleshooting the CloudWatch Agent

Issue	Resolution
Agent won't start	Check `/opt/aws/amazon-cloudwatch-agent/logs/amazon-cloudwatch-agent.log`; verify IAM permissions
Metrics not appearing	Confirm the namespace in the config; check for typos in metric names
IMDSv2 hop limit error	Increase the IMDSv2 hop limit on the instance to 2 for containers
Configuration not applied	Re-run `AmazonCloudWatch-ManageAgent` with action `configure`

> *Source: Troubleshooting the CloudWatch agent*

Skill 1.1.3 – Configure, Identify, and Troubleshoot CloudWatch Alarms

A **CloudWatch alarm** watches a single metric (or the result of a math expression) over a specified time period and performs one or more actions based on the value relative to a threshold.

> *Source: Metrics concepts – Alarms – Amazon CloudWatch*

Alarm States

State	Meaning
`OK`	The metric is within the defined threshold
`ALARM`	The metric has breached the threshold
`INSUFFICIENT_DATA`	Not enough data to determine the state (e.g., metric not yet published)

Alarm Actions

When an alarm transitions to `ALARM`, `OK`, or `INSUFFICIENT_DATA`, it can trigger:

- **Amazon SNS notification** – Send an email, SMS, or invoke a Lambda function.
- **EC2 action** – Stop, terminate, reboot, or recover an EC2 instance.
- **Auto Scaling action** – Scale in or scale out an Auto Scaling group.
- **Systems Manager OpsCenter** – Create an OpsItem for investigation.
- **EventBridge** – Route the alarm state change to any EventBridge target.

Composite Alarms

A **composite alarm** combines multiple alarms using Boolean logic (`AND`, `OR`, `NOT`). It reduces alarm noise by only triggering when a combination of conditions is true.

```
ALARM("HighCPU") AND ALARM("HighMemory")
```

Composite alarms can only trigger SNS notifications or other composite alarms — they cannot directly trigger EC2 or Auto Scaling actions.

Alarm Math Expressions

CloudWatch supports metric math expressions in alarms. For example, to alarm on error rate:

```
errorRate = errors / requests * 100
```

Anomaly Detection Alarms

CloudWatch can use machine learning to create a dynamic threshold band based on historical metric patterns. An anomaly detection alarm fires when the metric falls outside the expected band, accounting for time-of-day and day-of-week patterns.

Troubleshooting Alarms

Issue	Cause	Resolution
Alarm stuck in `INSUFFICIENT_DATA`	Metric not being published	Verify the agent is running; check the metric namespace and dimensions
Alarm not triggering	Threshold too high or evaluation period too long	Review the threshold and `DatapointsToAlarm` setting
Too many alarm notifications	Missing composite alarm	Create a composite alarm to reduce noise
Alarm action not executing	Missing IAM permissions	Ensure the alarm has permission to publish to SNS or invoke the target

---

Skill 1.1.4 – Create and Manage CloudWatch Dashboards

CloudWatch dashboards are customizable, shareable pages in the CloudWatch console that display metrics and alarms for AWS resources across multiple accounts and Regions.

> *Source: CloudWatch Dashboards – Amazon CloudWatch*

Dashboard Widget Types

Widget Type	Use Case
Line	Visualize metric trends over time
Stacked area	Show cumulative contribution of multiple metrics
Number	Display the latest value of a metric
Gauge	Show a metric value relative to a min/max range
Bar	Compare metric values across dimensions
Pie	Show proportional breakdown
Alarm status	Display the current state of one or more alarms
Log table	Display results of a CloudWatch Logs Insights query
Text	Add Markdown-formatted notes or documentation

Cross-Account and Cross-Region Dashboards

CloudWatch supports dashboards that display metrics from multiple AWS accounts and Regions. To enable this:

1. In the **monitoring account**, enable cross-account observability in CloudWatch settings.
2. In each **source account**, link the account to the monitoring account.
3. Once linked, the monitoring account can view metrics, logs, and alarms from all source accounts in a single dashboard.

Sharing Dashboards

Dashboards can be shared publicly (via a link) or with specific AWS accounts. Shared dashboards are read-only for recipients. You can optionally require a username and password for public shares.

Automatic Dashboards

CloudWatch provides pre-built **automatic dashboards** for many AWS services (EC2, RDS, Lambda, S3, etc.) that are populated automatically when you use those services. These require no configuration and are a good starting point for operational visibility.

Skill 1.1.5 – Configure AWS Services to Send Notifications to Amazon SNS

Amazon Simple Notification Service (Amazon SNS) is a fully managed pub/sub messaging service. Many AWS services can publish notifications to SNS topics, which then fan out to subscribers (email, SMS, Lambda, SQS, HTTP endpoints).

Common SNS Integration Patterns

AWS Service	How It Sends to SNS
CloudWatch Alarms	Alarm action: publish to SNS topic on state change
CloudTrail	Deliver log file notifications to SNS when new logs are written to S3
AWS Config	Notify on configuration changes and compliance evaluations
RDS	Event subscriptions for DB instance events (failover, backup, etc.)
Auto Scaling	Lifecycle hook notifications and scaling activity notifications
S3	Event notifications for object creation, deletion, etc.
AWS Budgets	Alert when cost or usage thresholds are exceeded

Creating an SNS-Backed CloudWatch Alarm

```
# 1. Create an SNS topic
  aws sns create-topic --name ops-alerts

  # 2. Subscribe an email address
  aws sns subscribe \
    --topic-arn arn:aws:sns:us-east-1:123456789012:ops-alerts \
    --protocol email \
    --notification-endpoint ops-team@example.com

  # 3. Create a CloudWatch alarm that publishes to the topic
  aws cloudwatch put-metric-alarm \
    --alarm-name HighCPUUtilization \
    --metric-name CPUUtilization \
    --namespace AWS/EC2 \
    --dimensions Name=InstanceId,Value=i-1234567890abcdef0 \
    --statistic Average \
    --period 300 \
    --threshold 80 \
    --comparison-operator GreaterThanThreshold \
    --evaluation-periods 2 \
    --alarm-actions arn:aws:sns:us-east-1:123456789012:ops-alerts
```

SNS Message Filtering

SNS supports **message filtering** using subscription filter policies. This allows a single topic to serve multiple subscribers, each receiving only the messages relevant to them — reducing the need for multiple topics.

Task 1.2: Identify and Remediate Issues Using Monitoring and Availability Metrics

Skill 1.2.1 – Analyze Performance Metrics and Automate Remediation

Effective CloudOps requires not just collecting metrics, but acting on them automatically. AWS provides a layered approach: CloudWatch detects anomalies, EventBridge routes events, and Lambda or Systems Manager executes remediation.

AWS Compute Optimizer

AWS Compute Optimizer analyzes CloudWatch utilization metrics for EC2 instances, Auto Scaling groups, EBS volumes, Lambda functions, and ECS services on Fargate. It uses machine learning to identify over-provisioned and under-provisioned resources and provides rightsizing recommendations.

> *Source: What is AWS Compute Optimizer? – AWS Compute Optimizer*

Key findings:

- **Over-provisioned** – The resource is larger than needed; downsizing will reduce cost.
- **Under-provisioned** – The resource is too small; performance may be degraded.
- **Optimized** – The resource is appropriately sized.
- **Insufficient data** – Not enough CloudWatch metrics to make a recommendation (requires at least 30 hours of data).

AWS User Notifications

AWS User Notifications provides a centralized service for configuring and viewing notifications from AWS services. It aggregates notifications from CloudWatch, AWS Health, Security Hub, and other services into a single console, and can deliver them to email, AWS Chatbot (Slack/Teams), or the AWS Console Mobile App.

Automated Remediation Patterns

Pattern	Services Used	Description
Alarm → SNS → Lambda	CloudWatch, SNS, Lambda	Alarm triggers SNS; SNS invokes Lambda to execute remediation logic
Alarm → EventBridge → SSM Automation	CloudWatch, EventBridge, Systems Manager	Alarm state change event triggers an SSM Automation runbook
CloudTrail → EventBridge → Lambda	CloudTrail, EventBridge, Lambda	Detect a specific API call and automatically respond
Config Rule → SSM Automation	AWS Config, Systems Manager	Non-compliant resource triggers an automatic remediation runbook
Auto Scaling	CloudWatch, Auto Scaling	Scale EC2 capacity automatically based on metric thresholds

Example: Auto-Remediate a Stopped EC2 Instance

```
CloudWatch Alarm (StatusCheckFailed)
    → SNS Topic
    → Lambda Function
    → EC2 RebootInstances API
```

Or using Systems Manager Automation directly:

```
CloudWatch Alarm (StatusCheckFailed_Instance)
    → Alarm Action: SSM Automation
    → Runbook: AWS-RestartEC2Instance
```

Skill 1.2.2 – Use EventBridge to Route, Enrich, and Deliver Events

Amazon EventBridge is a serverless event bus service that connects AWS services, SaaS applications, and custom applications using events. It is the backbone of event-driven automation in AWS.

> *Source: What Is Amazon EventBridge? – Amazon EventBridge*

Core EventBridge Concepts

Concept	Description
Event bus	A pipeline that receives events. The `default` event bus receives events from AWS services. Custom buses receive events from your applications. Partner buses receive events from SaaS providers.
Event	A JSON object representing a change in state (e.g., an EC2 instance state change, a CloudTrail API call).
Rule	Matches incoming events against an event pattern and routes matching events to one or more targets.
Target	The resource that processes the event (e.g., Lambda, SQS, SNS, Step Functions, SSM Automation, ECS task).
Event pattern	A JSON filter that specifies which events a rule matches.
Input transformer	Customizes the event payload before it is sent to the target.
Archive	Stores events for replay. Useful for debugging and reprocessing.
EventBridge Pipes	Point-to-point integrations between a source (SQS, Kinesis, DynamoDB Streams) and a target, with optional filtering and enrichment.
EventBridge Scheduler	Creates scheduled tasks (cron or rate expressions) that invoke targets at specified times.

Event Pattern Examples

Match any EC2 instance state change to `stopped`:

```
{
    "source": ["aws.ec2"],
    "detail-type": ["EC2 Instance State-change Notification"],
    "detail": {
      "state": ["stopped"]
    }
  }
```

Match a specific CloudTrail API call (e.g., someone deletes a security group):

```
{
    "source": ["aws.cloudtrail"],
    "detail-type": ["AWS API Call via CloudTrail"],
    "detail": {
      "eventSource": ["ec2.amazonaws.com"],
      "eventName": ["DeleteSecurityGroup"]
    }
  }
```

Input Transformation

Use an **input transformer** to reshape the event payload before sending it to a target. This is useful for formatting SNS notifications or passing specific fields to Lambda.

```
{
    "inputPathsMap": {
      "instance": "$.detail.instance-id",
      "state": "$.detail.state"
    },
    "inputTemplate": "\"Instance <instance> changed state to <state>\""
  }
```

Cross-Account Event Routing

EventBridge supports sending events from one account's event bus to another account's event bus. The target account must grant permission via a resource-based policy on its event bus. This enables centralized event processing in a monitoring account.

Troubleshooting EventBridge Rules

Issue	Cause	Resolution
Rule not triggering	Event pattern does not match	Use the **Event Pattern Sandbox** in the console to test patterns against sample events
Target not invoked	IAM permissions missing	Ensure the EventBridge execution role has permission to invoke the target
Events being throttled	Target is rate-limited	Check `ThrottledRules` CloudWatch metric; add a dead-letter queue (DLQ) to capture failed events
Events not arriving from another account	Missing resource policy on target bus	Add a resource-based policy allowing the source account to `events:PutEvents`

Key CloudWatch Metrics for EventBridge

- `TriggeredRules` – Number of rules that matched incoming events.

- `ThrottledRules` – Number of rules that were throttled.
- `FailedInvocations` – Number of times a target invocation failed.
- `DeadLetterInvocations` – Number of events sent to the DLQ.

> *Source: Best practices when defining rules in Amazon EventBridge*

Skill 1.2.3 – Create and Run Systems Manager Automation Runbooks

AWS Systems Manager Automation lets you create and run *runbooks* — SSM documents of type `Automation` — that define a series of steps to perform on AWS resources. Runbooks can call AWS APIs, run scripts, invoke Lambda functions, and pause for manual approvals.

> *Source: AWS Systems Manager Automation*

Predefined Runbooks

AWS provides hundreds of predefined runbooks. Common examples:

Runbook	Description
`AWS-RestartEC2Instance`	Stops and starts an EC2 instance
`AWS-StopEC2Instance`	Stops a running EC2 instance
`AWS-StartEC2Instance`	Starts a stopped EC2 instance
`AWS-CreateSnapshot`	Creates an EBS snapshot
`AWSSupport-TroubleshootSSH`	Diagnoses and fixes SSH connectivity issues
`AWSSupport-ResetAccess`	Recovers access to an EC2 instance (SSH key injection or password reset)
`AWSSupport-TroubleshootCloudWatchAgent`	Diagnoses CloudWatch agent issues
`AWS-EnableS3BucketEncryption`	Enables default encryption on an S3 bucket

Custom Runbooks

Custom runbooks are authored in YAML or JSON. Each step uses an **action type** to define what it does:

Action Type	Description
`aws:runCommand`	Run a shell or PowerShell command on managed nodes
`aws:executeScript`	Run a Python or PowerShell script inline
`aws:invokeLambdaFunction`	Invoke a Lambda function
`aws:executeAwsApi`	Call any AWS API
`aws:waitForAwsResourceProperty`	Poll a resource property until a condition is met
`aws:approve`	Pause execution and wait for manual approval via SNS
`aws:changeInstanceState`	Start, stop, or terminate EC2 instances
`aws:branch`	Conditionally branch to different steps based on a variable

Example Custom Runbook: Restart and Verify EC2 Instance

```
schemaVersion: "0.3"
  description: "Restart an EC2 instance and verify it is running"
  parameters:
    InstanceId:
      type: String
      description: "The ID of the EC2 instance to restart"
  mainSteps:
    - name: StopInstance
      action: aws:changeInstanceState
      inputs:
        InstanceIds:
          - "{{ InstanceId }}"
        DesiredState: stopped

    - name: StartInstance
      action: aws:changeInstanceState
      inputs:
        InstanceIds:
          - "{{ InstanceId }}"
        DesiredState: running

    - name: VerifyRunning
      action: aws:waitForAwsResourceProperty
      inputs:
        Service: ec2
        Api: DescribeInstances
        InstanceIds:
          - "{{ InstanceId }}"
        PropertySelector: "$.Reservations[0].Instances[0].State.Name"
        DesiredValues:
          - running
```

Automation Execution Modes

Mode	Description
Simple execution	Run the runbook once against a single target
Rate control	Run against multiple targets with configurable concurrency and error thresholds
Multi-account and multi-Region	Run across accounts and Regions using AWS Organizations integration
Change Calendar	Restrict automation execution to approved time windows

Triggering Automation

Runbooks can be triggered:

- **Manually** – From the Systems Manager console or CLI.
- **From a CloudWatch Alarm** – As an alarm action.
- **From EventBridge** – As a rule target.
- **From AWS Config** – As an automatic remediation action for non-compliant resources.
- **On a schedule** – Using Systems Manager Maintenance Windows.

Troubleshooting Automation

Error	Cause	Resolution
`IAM PassRole` error	The execution role lacks `iam:PassRole`	Add `iam:PassRole` to the role used to start the automation
VPC 400 error	The automation instance cannot reach SSM endpoints	Add VPC endpoints for `ssm`, `ssmmessages`, and `ec2messages`
Timeout	A step exceeded its `timeoutSeconds`	Increase `timeoutSeconds` for the affected step
`AssumeRole` error	The automation role cannot be assumed	Verify the trust policy on the automation role

> *Source: Troubleshooting Systems Manager Automation*

Task 1.3: Implement Performance Optimization Strategies for Compute, Storage, and Database Resources

Skill 1.3.1 – Optimize Compute Resources and Remediate Performance Problems

Identifying Performance Problems

The first step in compute optimization is identifying the right metrics. Key CloudWatch metrics for EC2:

Metric	Namespace	Description
`CPUUtilization`	`AWS/EC2`	Percentage of allocated EC2 compute units in use
`NetworkIn` / `NetworkOut`	`AWS/EC2`	Bytes transferred in/out of the instance
`DiskReadOps` / `DiskWriteOps`	`AWS/EC2`	Instance store disk operations (not EBS)
`StatusCheckFailed_Instance`	`AWS/EC2`	Instance-level status check failure
`StatusCheckFailed_System`	`AWS/EC2`	Host-level status check failure (requires AWS intervention)
`mem_used_percent`	`CWAgent`	Memory utilization (requires CloudWatch agent)
`disk_used_percent`	`CWAgent`	Disk utilization (requires CloudWatch agent)

AWS Compute Optimizer Recommendations

Compute Optimizer analyzes 14 days of CloudWatch metrics and provides recommendations for:

- **EC2 instance type** – Right-size to a smaller or different instance family.
- **Graviton migration** – Identify instances that can migrate to ARM-based Graviton instances for better price/performance.
- **Auto Scaling group** – Optimize the instance type used in the group.

Resource Tags for Cost and Performance Attribution

Use consistent resource tags (e.g., `Environment`, `Application`, `Team`) to:

- Filter CloudWatch metrics by tag using **CloudWatch Metrics Insights**.
- Attribute costs to teams using AWS Cost Explorer.
- Target Systems Manager operations (Run Command, Patch Manager) by tag.

EC2 Instance Types and Families

Family	Optimized For
`t` (e.g., t3, t4g)	Burstable general-purpose workloads
`m` (e.g., m6i, m7g)	Balanced compute, memory, and networking
`c` (e.g., c6i, c7g)	Compute-intensive workloads
`r` (e.g., r6i, r7g)	Memory-intensive workloads
`i` (e.g., i4i)	Storage-optimized with NVMe SSD
`p` / `g` (e.g., p4d, g5)	GPU-accelerated ML and graphics
`hpc` (e.g., hpc7g)	High-performance computing with EFA

Burstable Instance CPU Credits

`t` family instances earn CPU credits when idle and spend them when bursting above the baseline. Monitor `CPUCreditBalance` and `CPUSurplusCreditsCharged` to detect credit exhaustion. If an instance consistently exhausts credits, switch to a fixed-performance instance type or enable **unlimited mode** (which allows sustained bursting at an additional cost).

Skill 1.3.2 – Analyze Amazon EBS Performance Metrics and Optimize Volume Types

Amazon Elastic Block Store (Amazon EBS) provides persistent block storage for EC2 instances. Choosing the right volume type and monitoring the right metrics is critical for both performance and cost.

> *Source: Amazon EBS volume performance – Amazon EBS*

EBS Volume Types

Volume Type	Category	Max IOPS	Max Throughput	Use Case
`gp3`	SSD	16,000	1,000 MB/s	General-purpose; default choice
`gp2`	SSD	16,000	250 MB/s	Legacy general-purpose (prefer gp3)
`io2` / `io2 Block Express`	SSD	256,000	4,000 MB/s	Mission-critical databases (Oracle, SQL Server)
`io1`	SSD	64,000	1,000 MB/s	Legacy provisioned IOPS (prefer io2)
`st1`	HDD	500	500 MB/s	Throughput-intensive sequential workloads (log processing, big data)
`sc1`	HDD	250	250 MB/s	Cold data, infrequently accessed

Key difference between gp2 and gp3: `gp3` decouples IOPS and throughput from volume size, allowing you to provision up to 16,000 IOPS and 1,000 MB/s independently. `gp2` ties IOPS to volume size (3 IOPS/GB), making it more expensive to achieve high IOPS on smaller volumes.

Key CloudWatch Metrics for EBS

> *Source: Amazon CloudWatch metrics for Amazon EBS*

Metric	Description
`VolumeReadOps` / `VolumeWriteOps`	Total IOPS consumed
`VolumeReadBytes` / `VolumeWriteBytes`	Total throughput consumed
`VolumeTotalReadTime` / `VolumeTotalWriteTime`	Total time for I/O operations (used to calculate latency)
`VolumeQueueLength`	Number of pending I/O requests; high values indicate I/O bottleneck
`BurstBalance`	Remaining burst credits for `gp2` and `st1`/`sc1` volumes
`VolumeIdleTime`	Time the volume is not processing I/O

Calculating latency:

```
Average Read Latency = VolumeTotalReadTime / VolumeReadOps
```

EBS-Optimized Instances

EBS-optimized instances provide a dedicated network path between the instance and EBS, eliminating contention with regular network traffic. Most modern instance types are EBS-optimized by default. Always use EBS-optimized instances for I/O-intensive workloads.

Troubleshooting EBS Performance

Symptom	Likely Cause	Resolution
High `VolumeQueueLength`	IOPS limit reached	Upgrade to `io2` or increase provisioned IOPS on `gp3`
`BurstBalance` depleted	Sustained I/O on `gp2` volume	Migrate to `gp3` and provision IOPS independently
High latency	Volume type mismatch	Switch to SSD-backed volume; check for noisy neighbor on shared host
Throughput capped	Instance network bandwidth limit	Upgrade instance type; check EBS-optimized bandwidth limits

Modifying EBS Volumes (Elastic Volumes)

You can modify an EBS volume's type, size, IOPS, and throughput **without detaching it** using the **Elastic Volumes** feature. The modification takes effect immediately, though performance optimization may take up to 24 hours.

```
aws ec2 modify-volume \
    --volume-id vol-1234567890abcdef0 \
    --volume-type gp3 \
    --iops 6000 \
    --throughput 500
```

Skill 1.3.3 – Implement and Optimize Amazon S3 Performance Strategies

Amazon S3 is designed for high throughput and scales automatically. However, specific strategies can further optimize performance for large-scale workloads.

> *Source: Best practices design patterns: optimizing Amazon S3 performance*

S3 Request Rate Scaling

S3 automatically scales to handle high request rates. Each S3 prefix supports:

- **3,500 PUT/COPY/POST/DELETE requests per second**
- **5,500 GET/HEAD requests per second**

To scale beyond these limits, distribute objects across **multiple prefixes**. For example, instead of all objects under `photos/`, use `photos/a/`, `photos/b/`, `photos/c/`, etc.

S3 Transfer Acceleration

S3 Transfer Acceleration speeds up uploads to S3 by routing data through AWS CloudFront edge locations using optimized network paths. It is most beneficial for:

- Uploads from geographically distant clients.
- Large file uploads over long distances.

> *Source: Configuring fast, secure file transfers using Amazon S3 Transfer Acceleration*

Enable Transfer Acceleration on a bucket:

```
aws s3api put-bucket-accelerate-configuration \
    --bucket my-bucket \
    --accelerate-configuration Status=Enabled
```

Use the accelerated endpoint: `my-bucket.s3-accelerate.amazonaws.com`

Multipart Upload

Multipart upload allows you to upload a single object as a set of parts. It is recommended for objects larger than 100 MB and required for objects larger than 5 GB.

Benefits:

- **Improved throughput** – Upload parts in parallel.
- **Resilience** – If a part fails, retry only that part.
- **Pause and resume** – Upload can be paused and resumed.

> *Source: Uploading and copying objects using multipart upload in Amazon S3*

Important: Incomplete multipart uploads consume storage and incur charges. Use an S3 Lifecycle rule to abort incomplete multipart uploads after a specified number of days:

```
{
    "Rules": [
      {
        "ID": "AbortIncompleteMultipartUploads",
        "Status": "Enabled",
        "AbortIncompleteMultipartUpload": {
          "DaysAfterInitiation": 7
        }
      }
    ]
  }
```

AWS DataSync

AWS DataSync is a managed data transfer service for moving large amounts of data between on-premises storage and AWS storage services (S3, EFS, FSx). It automates scheduling, monitoring, and data integrity validation.

Use DataSync when:

- Migrating data from on-premises NFS/SMB shares to S3 or EFS.
- Replicating data between AWS storage services.
- Transferring data at scale with built-in retry and verification.

S3 Lifecycle Policies

S3 Lifecycle policies automate the transition of objects between storage classes and the expiration of objects, reducing storage costs.

Storage Class	Use Case	Retrieval Time
S3 Standard	Frequently accessed data	Milliseconds
S3 Intelligent-Tiering	Unknown or changing access patterns	Milliseconds
S3 Standard-IA	Infrequently accessed, rapid retrieval	Milliseconds
S3 One Zone-IA	Infrequently accessed, single AZ	Milliseconds
S3 Glacier Instant Retrieval	Archive with millisecond access	Milliseconds
S3 Glacier Flexible Retrieval	Archive, 1–12 hour retrieval	Minutes to hours
S3 Glacier Deep Archive	Long-term archive, lowest cost	Up to 12 hours

Example lifecycle rule: transition to Standard-IA after 30 days, then to Glacier after 90 days:

```
{
    "Rules": [
      {
        "ID": "ArchiveOldObjects",
        "Status": "Enabled",
        "Transitions": [
          { "Days": 30, "StorageClass": "STANDARD_IA" },
          { "Days": 90, "StorageClass": "GLACIER" }
        ]
      }
    ]
  }
```

Byte-Range Fetches

For large objects, use **byte-range fetches** to download specific portions of an object in parallel, improving download throughput and enabling partial retrieval.

Skill 1.3.4 – Evaluate and Select Shared Storage Solutions (Amazon EFS, Amazon FSx)

Amazon Elastic File System (Amazon EFS)

Amazon EFS is a fully managed, elastic NFS file system for Linux workloads. It scales automatically from gigabytes to petabytes and supports concurrent access from thousands of EC2 instances across multiple Availability Zones.

> *Source: Amazon Elastic File System – User Guide*

EFS Performance Modes

Mode	Description	Use Case
General Purpose (default)	Lowest latency; up to 35,000 IOPS	Web serving, content management, home directories
Max I/O	Higher aggregate throughput; slightly higher latency	Big data, media processing, parallel workloads

EFS Throughput Modes

Mode	Description
Elastic (recommended)	Automatically scales throughput up and down based on workload; pay per GB transferred
Provisioned	Specify a fixed throughput level independent of storage size
Bursting (legacy)	Throughput scales with storage size; earns burst credits

EFS Storage Classes and Lifecycle Policies

Storage Class	Description
EFS Standard	Frequently accessed files; multi-AZ redundancy
EFS Standard-IA	Infrequently accessed files; lower cost, retrieval fee
EFS One Zone	Frequently accessed; single AZ (lower cost, less resilient)
EFS One Zone-IA	Infrequently accessed; single AZ

EFS Lifecycle policies automatically move files that have not been accessed for a specified period (7, 14, 30, 60, or 90 days) to the IA storage class, reducing costs. Files are moved back to Standard when accessed.

EFS Access Points

EFS Access Points are application-specific entry points into an EFS file system. They enforce a specific POSIX user identity and a root directory for all file system requests made through the access point, simplifying permissions management for containerized applications.

Amazon FSx

Amazon FSx provides fully managed file systems optimized for specific workloads. There are four FSx variants:

FSx Variant	Protocol	Use Case
FSx for Windows File Server	SMB	Windows applications, Active Directory integration, .NET apps
FSx for Lustre	Lustre	High-performance computing (HPC), ML training, financial modeling
FSx for NetApp ONTAP	NFS, SMB, iSCSI	Enterprise storage, multi-protocol access, data management
FSx for OpenZFS	NFS	Linux workloads requiring ZFS features (snapshots, clones)

FSx for Windows File Server

- Integrates with **Microsoft Active Directory** for user authentication.
- Supports **DFS Namespaces** for organizing shares across multiple file systems.
- Provides **Multi-AZ** deployment for high availability.
- Supports **shadow copies** (VSS snapshots) for file-level recovery.

FSx for Lustre

- Delivers sub-millisecond latencies and hundreds of GB/s of throughput.
- Can be linked to an **S3 bucket** for lazy loading: data is loaded from S3 on first access and written back to S3 on completion.
- Ideal for ML training jobs that need fast access to large datasets stored in S3.

Choosing Between EFS and FSx

Requirement	Recommended Service
Linux NFS, elastic scaling, multi-AZ	Amazon EFS
Windows SMB, Active Directory	FSx for Windows File Server
HPC, ML, high throughput	FSx for Lustre
Enterprise multi-protocol (NFS + SMB + iSCSI)	FSx for NetApp ONTAP
Linux NFS with ZFS features	FSx for OpenZFS

Skill 1.3.5 – Monitor Amazon RDS Metrics and Modify Configurations to Increase Performance

RDS CloudWatch Metrics

> *Source: Monitoring Amazon RDS metrics with Amazon CloudWatch*

Metric	Description
`CPUUtilization`	Percentage of CPU used by the DB instance
`DatabaseConnections`	Number of active connections to the DB
`FreeableMemory`	Available RAM in bytes
`FreeStorageSpace`	Available storage in bytes
`ReadIOPS` / `WriteIOPS`	Average IOPS for read/write operations
`ReadLatency` / `WriteLatency`	Average time per I/O operation
`ReadThroughput` / `WriteThroughput`	Average throughput in bytes/second
`ReplicaLag`	Lag between primary and read replica (seconds)
`SwapUsage`	Amount of swap space used (high values indicate memory pressure)

Amazon RDS Performance Insights

RDS Performance Insights is a database performance tuning and monitoring feature that visualizes database load and helps identify performance bottlenecks.

> *Source: Monitoring DB load with Performance Insights on Amazon RDS*

Key concepts:

- **DB Load** – The average number of active sessions. A DB load above the number of vCPUs indicates a bottleneck.
- **Average Active Sessions (AAS)** – The primary metric in Performance Insights.
- **Top SQL** – Identifies the SQL queries contributing most to DB load.
- **Wait events** – Shows what the database is waiting on (CPU, I/O, locks, etc.).
- **Proactive recommendations** – Performance Insights can automatically detect and recommend fixes for common performance issues (e.g., missing indexes, idle connections).

Note: Performance Insights is being upgraded to **CloudWatch Database Insights**. Existing Performance Insights users should plan to migrate to Database Insights Advanced mode before the end-of-life deadline.

Enhanced Monitoring

Enhanced Monitoring provides OS-level metrics (CPU, memory, file system, disk I/O) at up to 1-second granularity. Unlike CloudWatch metrics (which come from the hypervisor), Enhanced Monitoring metrics come from an agent running on the DB instance, providing more accurate memory and CPU data.

Enhanced Monitoring data is delivered to **CloudWatch Logs** (not CloudWatch Metrics) and can be viewed in the RDS console.

RDS Proxy

Amazon RDS Proxy is a fully managed database proxy that sits between your application and RDS. It pools and shares database connections, reducing the overhead of opening and closing connections for serverless or high-concurrency workloads.

> *Source: Amazon RDS Proxy – Amazon Relational Database Service*

Benefits:

- **Connection pooling** – Reduces the number of connections to the database, improving scalability.
- **Failover resilience** – RDS Proxy maintains connections during Multi-AZ failover, reducing failover time by up to 66%.
- **IAM authentication** – Supports IAM-based authentication, eliminating the need to embed database credentials in application code.
- **Secrets Manager integration** – Credentials are stored in Secrets Manager and rotated automatically.

Use cases:

- Lambda functions connecting to RDS (Lambda can open thousands of concurrent connections).
- Applications with unpredictable connection spikes.
- Microservices architectures with many short-lived connections.

Performance Optimization Strategies for RDS

Strategy	Description
Read replicas	Offload read traffic from the primary instance to one or more read replicas
Multi-AZ	Provides high availability; standby replica is not used for reads
Instance class upgrade	Move to a larger instance class for more CPU and memory
Storage type	Use `io2` or `gp3` for I/O-intensive workloads
Parameter group tuning	Adjust database engine parameters (e.g., `innodb_buffer_pool_size` for MySQL)
Query optimization	Use Performance Insights to identify and optimize slow queries
RDS Proxy	Reduce connection overhead for serverless and high-concurrency workloads

---

Skill 1.3.6 – Implement, Monitor, and Optimize EC2 Instances and Associated Storage and Networking

EC2 Placement Groups

EC2 placement groups influence the placement of EC2 instances on the underlying hardware to meet specific performance or availability requirements.

> *Source: Placement groups for your Amazon EC2 instances – Amazon EC2*

Placement Group Strategies

Strategy	Description	Use Case
Cluster	Packs instances close together in a single AZ. Provides low-latency, high-bandwidth networking (up to 100 Gbps with EFA).	HPC, tightly coupled parallel computing, ML training
Partition	Divides instances into logical partitions, each on separate racks with independent power and networking. Up to 7 partitions per AZ.	Distributed databases (Cassandra, HDFS, Kafka)
Spread	Places each instance on distinct hardware (separate racks). Maximum 7 instances per AZ per group.	Small groups of critical instances that must be isolated from each other

Elastic Fabric Adapter (EFA)

EFA is a network interface for EC2 instances that provides OS-bypass networking for HPC and ML workloads. It delivers the low latency and high throughput of on-premises HPC clusters. EFA is used with cluster placement groups for maximum performance.

EC2 Instance Storage Optimization

Storage Type	Characteristics	Best For
EBS (gp3)	Persistent, network-attached, flexible IOPS/throughput	General-purpose workloads, databases
EBS (io2)	Persistent, high IOPS, 99.999% durability	Mission-critical databases
Instance store (NVMe SSD)	Ephemeral, physically attached, extremely low latency	Temporary data, caches, buffers
EFS	Shared NFS, elastic, multi-AZ	Shared file storage across multiple instances

EC2 Network Performance

EC2 network performance scales with instance size. Key networking features:

- **Enhanced Networking (ENA)** – Provides higher bandwidth, lower latency, and lower jitter using the Elastic Network Adapter. Enabled by default on most modern instance types.
- **Elastic Fabric Adapter (EFA)** – For HPC workloads requiring MPI or NCCL communication.
- **Network bandwidth limits** – Each instance type has a maximum network bandwidth. Monitor `NetworkIn` / `NetworkOut` and compare against the instance's documented limit.

Monitoring EC2 with CloudWatch

For comprehensive EC2 monitoring, combine:

1. **Default EC2 metrics** (hypervisor-level: CPU, network, disk I/O) — available at 5-minute intervals by default; enable **detailed monitoring** for 1-minute intervals.
2. **CloudWatch agent metrics** (OS-level: memory, disk space, swap) — requires agent installation.
3. **Application-level metrics** — Publish custom metrics from your application using the CloudWatch PutMetricData API or the embedded metric format (EMF).

AWS Compute Optimizer for EC2

Compute Optimizer analyzes EC2 utilization metrics and recommends:

- Downsizing over-provisioned instances.
- Upsizing under-provisioned instances.
- Migrating to Graviton (ARM) instances for better price/performance.
- Switching to a different instance family better suited to the workload type.

Enable Compute Optimizer at the organization level to get recommendations across all accounts.

Summary Reference Table

Service	Primary Use in Domain 1
Amazon CloudWatch	Metrics, alarms, dashboards, logs, anomaly detection
AWS CloudTrail	API audit logging, compliance, security investigation
Amazon Managed Service for Prometheus	Kubernetes/container metrics with PromQL
CloudWatch Agent	OS-level metrics and log collection from EC2/ECS/EKS
Amazon EventBridge	Event routing, automation triggers, cross-account events
AWS Systems Manager Automation	Runbook-based remediation, operational automation
AWS Compute Optimizer	EC2/EBS/Lambda rightsizing recommendations
Amazon SNS	Notifications from alarms, Config, RDS, Auto Scaling
Amazon EBS	Block storage; optimize with gp3, io2, Elastic Volumes
Amazon S3	Object storage; optimize with Transfer Acceleration, multipart, lifecycle
Amazon EFS	Shared NFS; optimize with lifecycle policies, Elastic throughput
Amazon FSx	Managed file systems for Windows, HPC, enterprise workloads
Amazon RDS Performance Insights	Database load analysis, slow query identification
Amazon RDS Proxy	Connection pooling, failover resilience for RDS
EC2 Placement Groups	Cluster (HPC), Partition (distributed DB), Spread (HA)

Sources:

- Amazon CloudWatch Documentation
- AWS CloudTrail Documentation
- Amazon Managed Service for Prometheus Documentation
- Amazon EventBridge Documentation
- AWS Systems Manager Documentation
- Amazon EBS Documentation
- Amazon S3 Performance Documentation
- Amazon EFS Documentation
- Amazon RDS Documentation
- Amazon EC2 Placement Groups
- AWS Compute Optimizer Documentation